



Curso:

Matemáticas discretas 2

Práctica 1

Estudiante

Nathan Josue Esquivel Mendez

Fabiola Román Barrantes

Cristopher Gutierrez Vega

Kenneth Chaves Coto

Profesor:

Rebeca Paola Chavez Morales

Fecha:

22 septiembre 2026

1. Cómo se resolvió el reto

El desarrollo se dividió en tres etapas que reflejan directamente lo que pedía el enunciado.

Primero se implementó el criptosistema ElGamal: se definieron un primo p y un generador g públicos, se generó una llave privada x al azar, y se calculó la llave pública $y = g^x \bmod p$. Con esto, cada mensaje m se cifra eligiendo un número aleatorio k y calculando el par $E(m) = (c_1, c_2) = (g^k \bmod p, m \cdot y^k \bmod p)$.

Segundo, se construyó la parte más particular del reto: una Máquina de Turing simulada que recibe únicamente los dos cifrados nunca los mensajes originales y realiza la multiplicación modular directamente sobre ellos. Para esto, cada componente del cifrado se representó como una cinta de celdas en base 256 (bloques de 0 a 255), y se programó un algoritmo de multiplicación larga con acarreo (igual al método que se aprende en la escuela, pero en base 256) para multiplicar los componentes c_1 entre sí y los componentes c_2 entre sí. Como el resultado de esa multiplicación puede superar el valor de p , se aplicó después un algoritmo de reducción modular por desplazar y restar, que simula cómo un cabezal de Turing compararía y restaría valores celda por celda hasta obtener el residuo correcto.

Tercero, se verificó la propiedad homomórfica: el cifrado resultante de la Máquina de Turing se descifra usando la llave privada x , y ese valor se comparó contra el producto $m_1 \times m_2 \bmod p$ calculado directamente sobre los mensajes originales. Al coincidir ambos valores, quedó demostrado que multiplicar los cifrados componente a componente equivale a cifrar el producto de los mensajes, sin que en ningún momento el algoritmo "supiera" qué números estaba procesando.

2. Principales dificultades

La primera dificultad fue puramente conceptual: entender cómo es posible que una operación matemática realizada sobre datos cifrados produzca, al descifrarse, el resultado correcto de esa misma operación sobre los datos originales. No es un comportamiento intuitivo, y antes de poder programar algo hubo que entender por qué esta propiedad existe en ElGamal específicamente y no en cualquier esquema de cifrado.

La segunda dificultad, y probablemente la más exigente, fue construir la demostración formal de la propiedad homomórfica. No bastaba con que el código funcionara: había que justificar paso a paso por qué $E(m_1) \cdot E(m_2)$ es igual a $E(m_1 \cdot m_2 \bmod p)$, agrupando los exponentes de g y viendo cómo, al descifrar, los términos con la llave pública y se cancelan exactamente con los términos de c_1 elevados a x . Llegar a ese punto requirió escribir la demostración varias veces en papel antes de que las cancelaciones tuvieran sentido completo.

La tercera dificultad fue de representación más que de matemáticas: simular la Máquina de Turing exigió dejar de pensar los números como valores abstractos y empezar a pensarlos como secuencias de celdas en una cinta. Diseñar la multiplicación larga en base 256 con acarreo entre celdas, y sobre todo el algoritmo de reducción módulo p por desplazar y restar (en lugar de usar directamente el operador de residuo), fue un ejercicio distinto a lo habitual, porque la meta no era solo obtener el resultado correcto sino que cada paso reflejara

honestamente un proceso mecánico, sin “atajos” que rompieran la idea de que el servidor nunca interpreta los datos.

Por último, hubo que anticipar y documentar los casos de borde del sistema el mensaje 0, la existencia garantizada de inversos en Z_p por ser p primo, la generación válida del número aleatorio k y el desbordamiento de celdas al multiplicar para que la implementación no solo funcionará con los ejemplos de prueba, sino que estuviera justificada matemáticamente en los extremos.

3. Cómo se implementaron los conocimientos vistos en clase

Congruencias numéricas y aritmética modular: toda la seguridad y el funcionamiento de ElGamal dependen de trabajar dentro de Z_p , usando exponenciación modular rápida y el algoritmo extendido de Euclides para calcular inversos multiplicativos, ambos conceptos de congruencias aplicados directamente en código.

Sistemas de numeración en bloques (base 256): se aplicó al representar cada componente del cifrado como un arreglo de dígitos en base 256 en lugar de base 10, y al implementar la conversión entre un número y su representación en bloques, tal como se estudia el cambio de base numérica.

Propiedad multiplicativa homomórfica de ElGamal: se implementó tanto en código (la multiplicación componente a componente de los cifrados) como en la demostración matemática formal que explica por qué esa operación es válida.

*Álgebra y grupos (Z_p):** el hecho de que p sea primo garantiza que todo elemento distinto de cero tenga inverso, propiedad de teoría de grupos que se usó para justificar por qué no existen elementos sin inverso en el sistema, salvo el caso límite del cero.

4. Explicacion del codigo

Parte 1

Esta parte define las bases del cifrado: los parámetros públicos, las funciones matemáticas que todo lo demás va a usar, y la generación de llaves.

`const p = 467, g = 2` Son los parámetros públicos del sistema: p es un número primo (define el "tamaño" del mundo donde se hacen todas las operaciones) y g es un generador de Z_p^* . Se ponen fijos y conocidos por todos porque en ElGamal la seguridad no depende de ocultar p ni g , sino de ocultar la llave privada x .

`let x = 0, y = 0, E1 = null, E2 = null, Eres = null` Son las variables globales que van a guardar el estado del programa: `x` es la llave privada, y la llave pública, `E1` y `E2` los dos mensajes ya cifrados, y `Eres` el resultado que va a devolver la Máquina de Turing más adelante. Se declaran vacías porque todavía no se ha generado nada.

`modpow(b, e, m)` Calcula $b^e \bmod m$ de forma rápida (exponenciación modular por cuadrados). Se necesita porque calcular algo como $g^k \bmod p$ con exponentes grandes elevando primero y sacando el módulo después haría que el número creciera muchísimo; este método va sacando el módulo en cada paso para que nunca se dispare.

`egcd(a, b)` Es el algoritmo extendido de Euclides. Además del máximo común divisor, devuelve los coeficientes que permiten construir el inverso multiplicativo de un número. Se necesita porque más adelante, para desenscriptar, hay que "dividir" en aritmética modular, y ahí no existe la división normal: se usa el inverso multiplicativo en su lugar.

`modinv(a, m)` Usa `egcd` para obtener directamente el inverso de `a` módulo `m`. Es la pieza que va a permitir desenscriptar el mensaje al final del proceso.

`genKeys()` Es la función que arranca todo el sistema:

- Genera `x` al azar (la llave privada, que solo debe conocer el cliente).
- Calcula $y = g^x \bmod p$ (la llave pública, la que se comparte).
- Actualiza la pantalla para mostrar `x` y `y`.
- Habilita el botón de cifrar, porque ya hay llaves listas.
- Resetea todas las variables y la interfaz (cifrados anteriores, el registro de la Máquina de Turing, el resultado de verificación) para que, si el usuario genera llaves nuevas, no queden datos viejos mezclados con los nuevos.

Parte 2

Esta parte toma los parámetros y las llaves que ya se generaron en la Parte 1, y cifra los dos mensajes que se quieren multiplicar, dejándolos listos para que la Máquina de Turing opere sobre ellos sin conocerlos.

`const k = 2 + Math.floor(Math.random() * (p - 3))` Es el número aleatorio que se genera cada vez que se cifra un mensaje. Cambia en cada cifrado, por eso cifrar el mismo número dos veces da resultados distintos — esto es lo que le da seguridad semantica al esquema.

`c1: modpow(g, k, p)` Es una especie de huella de ese `k`, calculada con el generador `g`. Sirve para que, más adelante, quien tenga la llave privada `x` pueda deshacer el cifrado.

$c_2: (m * \text{modpow}(y, k, p)) \% p$ Es el mensaje real, multiplicado por la llave pública elevada a k . Este es el valor que realmente “esconde” a m ; sin la llave privada, no dice nada sobre el mensaje original.

El par (c_1, c_2) que devuelve $\text{encrypt}(m)$ es el cifrado completo $E(m)$. A partir de aquí, el resto del programa solo trabaja con estos dos números, nunca con m directamente.

$\text{encryptBoth}()$ Es la función que cifra los dos mensajes que el usuario escribió en pantalla:

- Lee m_1 y m_2 de los campos de entrada.
- Valida que ambos estén entre 0 y $p-1$, porque toda la aritmética del sistema ocurre en ese rango.
- Cifra cada uno llamando a $\text{encrypt}()$, y guarda los resultados en E_1 y E_2 .
- Muestra ambos cifrados en pantalla.
- Habilita el botón de la Máquina de Turing, porque ya hay algo cifrado sobre lo cual operar.
- Resetea cualquier resultado o verificación anterior, para que un cifrado nuevo no deje en pantalla datos de una corrida vieja.

Concepto matemático: congruencias y exponenciación modular; el valor aleatorio k es lo que le da seguridad al cifrado.

Parte 3

Esta parte contiene todo lo que sería la parte de la máquina de turing

Lo primero que podemos observar es la conversión a base 256

```
function toBase256(n) {
  const d = [];
  if (n === 0) d.push(0);
  while (n > 0) { d.push(n % 256); n = Math.floor(n / 256); }
  return d;
}
```

esto matemáticamente se puede representar así

$$n = d_0 + d_1 \cdot 256 + d_2 \cdot 256^2 + \dots$$

El ciclo while del código va sacando los dígitos uno por uno y mientras que el el Math descompone, la otra función que tenemos hace los mismo nada mas que es en base 256 en lugar de base 2

Después seguimos con tmMultiply

```
for (let i = 0; i < aD.length; i++) {
  let carry = 0;
  for (let j = 0; j < bD.length; j++) {
    const pos = i + j, prod = aD[i] * bD[j] + res[pos] + carry;
    res[pos] = prod % 256; carry = Math.floor(prod / 256);
    ...
  }
}
```

esto implementa

$$a \cdot b = \left(\sum_i a_i \cdot 256^i \right) \left(\sum_j b_j \cdot 256^j \right) = \sum_{i,j} a_i b_j \cdot 256^{i+j}$$

Por eso $a[i] \cdot b[j]$ se suma en la posición $pos = i+j$ — porque $256^i \cdot 256^j = 256^{i+j}$, exactamente igual que cuando multiplicas "23 × 45" y el producto de las unidades por las decenas cae en la columna de las decenas.

Como $a[i] \cdot b[j]$ puede superar 255 (el máximo que cabe en un byte), separas el resultado en dígito ($\% 256$) y acarreo ($/ 256$), igual que cuando en una multiplicación a mano "te llevas 1"

Seguimos con el tmModReduce este se encarga de sacar el residuo mod p sin usar %

Esta es la parte más bonita matemáticamente. En vez de pedirle a JavaScript el resto directo, la función simula cómo lo haría una máquina que solo sabe restar y dividir entre dos: es la división larga de toda la vida, pero en binario. Primero busca la copia más grande

del primo p (multiplicada por una potencia de dos) que todavía quepa dentro del número. Después va bajando de escala, dividiendo esa copia entre dos en cada paso, y en cada nivel pregunta si todavía cabe en lo que queda: si cabe, resta; si no, sigue de largo. Es el mismo razonamiento de dividir a mano probando "¿cabe cien veces? ¿diez veces? ¿una vez?", solo que aquí las escalas son potencias de dos. Al final, lo que sobra es el residuo que buscaba.

```
let shift = 0, shifted = mod;
while (shifted * 2 <= n) { shifted *= 2; shift++; }
```

Seguimos con el runTM

```
const prod1 = tmMultiply(c1_1, c1_2, log);
const c1res = tmModReduce(prod1, p, log);
...
const prod2 = tmMultiply(c2_1, c2_2, log);
const c2res = tmModReduce(prod2, p, log);
Eres = { c1: c1res, c2: c2res };
```

Esta función no hace matemática nueva, solo junta las dos piezas anteriores multiplicar y reducir y las aplica dos veces, una por cada mitad del mensaje cifrado. La magia está en que ElGamal tiene una propiedad especial: si multiplicás por separado las dos mitades de dos cifrados y después le sacás el resto módulo el primo del sistema, lo que obtenés es el cifrado del producto de los mensajes originales. La aritmética "atraviesa" el candado sin necesidad de abrirlo. runTM simplemente organiza ese proceso: toma las cuatro piezas cifradas, las pasa por la cinta simulada, y guarda el resultado que sigue cifrado para mostrarlo.

En ningún momento del recorrido se ve el mensaje original sin cifrar; la máquina solo toca números ya cifrados. Recién en la función de verificación, que viene después, se descifra el resultado —pero solo para comprobar que coincide con lo esperado, no porque haga falta para el cálculo.

Parte 4

Bueno, esta parte del código es la que usamos para verificar que el resultado de la operación sea correcto.

Primero, tomamos los dos valores que ingresó el usuario, que serían m_1 y m_2 . Después usamos el resultado cifrado para hacer la descryptación y obtener el resultado final.

Por otro lado, calculamos cuál debería ser el resultado normalmente, haciendo m_1 por m_2 mod p .

Entonces comparamos los dos resultados. Si el resultado que obtuvimos al descryptar es igual al resultado que calculamos normalmente, significa que la operación funcionó y que se está cumpliendo la propiedad homomórfica de ElGamal.

Por ejemplo, si tenemos 4 y 5, hacemos 4 por 5 y nos da 20. Si al descryptar también obtenemos 20, sabemos que todo salió bien.

Y la función `setVerify` simplemente se encarga de mostrar ese resultado en la pantalla, diciendo si la propiedad fue verificada o si los resultados no coinciden.

Finalmente, `genKeys()` genera las claves que necesitamos para trabajar con ElGamal.

5. Conclusiones

El reto permitió comprobar de forma práctica que es posible delegar un cálculo matemático a un tercero no confiable sin revelar los datos sobre los que opera, que es la idea central detrás del cómputo homomórfico y de aplicaciones reales como el voto electrónico cifrado o el procesamiento de datos sensibles en la nube.

Programar el criptosistema en sí resultó relativamente directo una vez comprendida la teoría, pero demostrar formalmente la propiedad homomórfica exigió un nivel de rigor matemático mayor al que suele requerir una tarea de programación convencional: no bastaba con que el código diera el resultado correcto, sino que cada paso debía estar justificado algebraicamente.

Simular la Máquina de Turing sobre bloques base 256 reforzó la relación entre estructuras de datos, representación numérica y algoritmos clásicos (multiplicación larga, división por desplazar y restar), mostrando que conceptos de bajo nivel como el manejo de acarreo siguen siendo relevantes incluso en un contexto criptográfico moderno.

En conjunto, el trabajo integró aritmética modular, teoría de números, estructuras de datos y demostración matemática formal en un solo sistema funcional, cumpliendo el objetivo de construir una "calculadora en la nube" que opera exclusivamente sobre datos cifrados.

